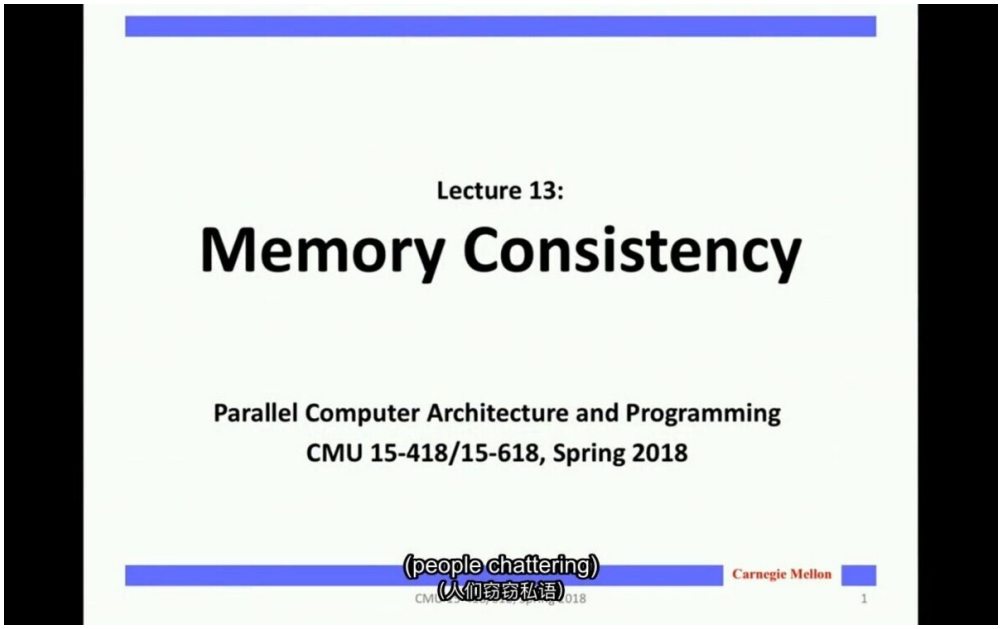
课程笔记

内存一致性模型：从顺序一致性到栅栏

CMU 15-418/618 并行计算机架构与编程 · Lecture 16（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 8 分 58 秒
视频链接：`https://www.bilibili.com/video/BV1Tj411z7U1/`

目录

1 问题：并行内存层次应该表现成什么样 **2**
1.1 为什么”读到最新值”不可行 . . . 2
1.2 本章小结 . . . 3

2 用一个实验建立顺序一致性的定义 **3**
2.1 实验设置 . . . 3
2.2 判断规则 . . . 3
2.3 对应的硬件想象 . . . 4
2.4 本章小结 . . . 5

3 横跨不同地址：更难的约束 **6**
3.1 顺序一致性的判据 . . . 6
3.2 单核上已经不成立 . . . 6
3.3 写缓冲区：重排的物理来源 . . . 7
3.4 本章小结 . . . 9

4 反例：标志位同步为什么会失效 **10**
4.1 经典模式 . . . 10
4.2 链式依赖也救不了 . . . 11
4.3 精简成可判定的反例 . . . 11
4.4 本章小结 . . . 12

5 如何恢复顺序：从严格到逐步放宽 **12**
5.1 最严格的做法 . . . 12
5.2 完成是什么意思 . . . 12
5.3 严格方案的代价 . . . 13
5.4 TSO：允许读越过写 . . . 14
5.5 PSO：连写也放宽 . . . 14
5.6 本章小结 . . . 15

6 弱序模型与栅栏 **15**
6.1 假设程序已正确同步 . . . 15
6.2 只读共享数据的额外麻烦 . . . 16
6.3 MFENCE . . . 16
6.4 ARM 的现实教训 . . . 17
6.5 本章小结 . . . 18

7 总结与延伸 **18**

1 问题：并行内存层次应该表现成什么样

上一讲讨论缓存协议时，我们只处理了单个缓存行内的一致性。这一讲要看更大的图景。讲者开场就说清了范围：之前的协议保证了某个缓存行上的读写按某种顺序一致的方式处理，但完全没有讨论 ** 不同地址之间 ** 的访问有没有约束。

如果没有约束，程序几乎无法写。

1.1 为什么”读到最新值”不可行

最直觉的定义是：每次读都拿到该地址最近被写入的值。

讲者说这在层次结构下不可行。有些操作一个时钟周期或更短就能完成，有些要几百个周期——从 1 个周期到 300 个周期，相差两个数量级。在一个大系统里，”时间”这个概念本身就不是严格对齐的。要求”这里的写必须立刻被别处的读看到”不切实际。

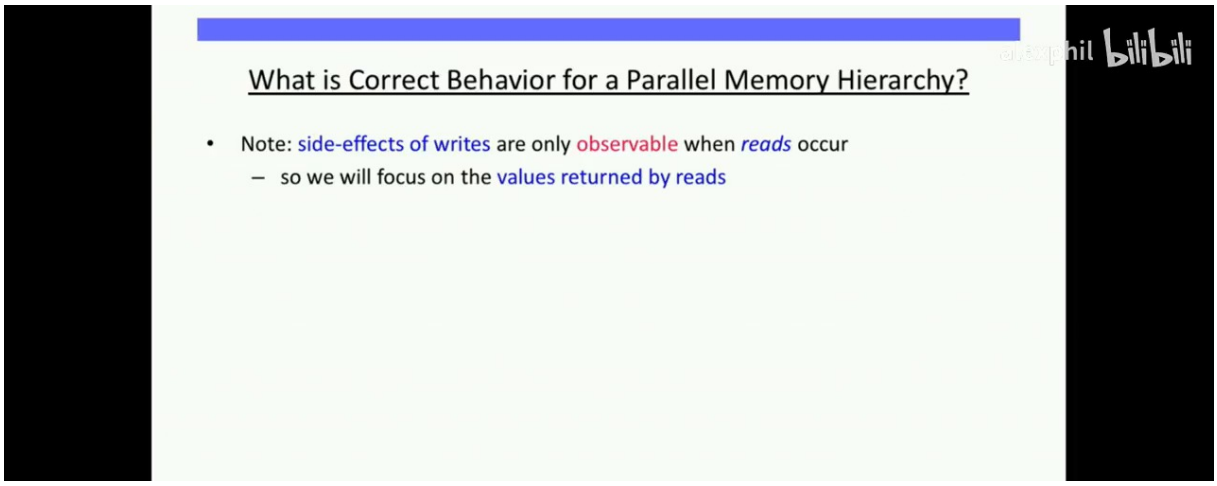


图 1: 并行内存层次的正确行为：写的副作用只在被观察时才有意义[1]

另一个极端是完全混乱：谁读谁写，能不能看到全凭运气。这会让软件开发变得极其困难。

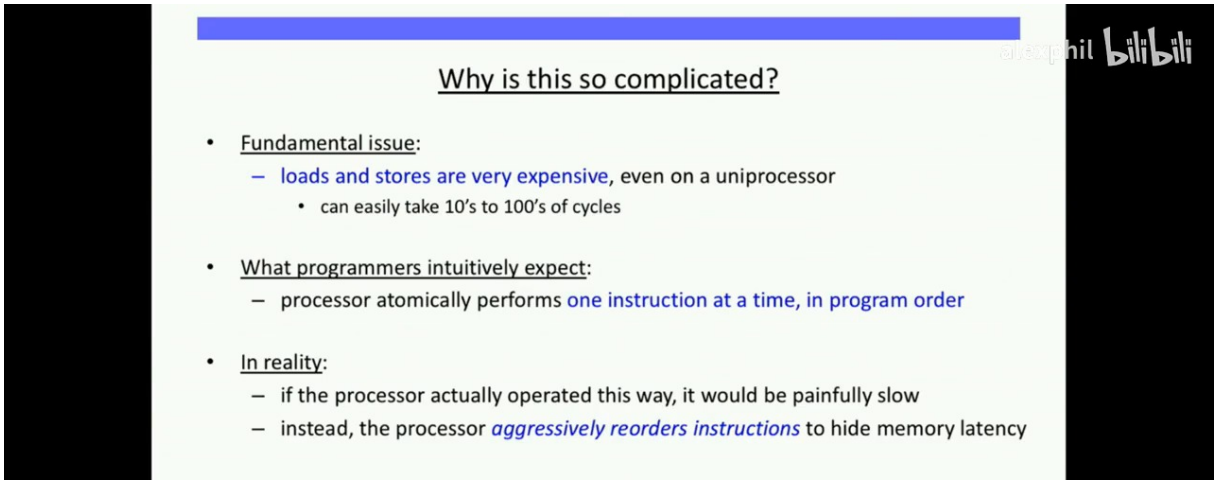


图 2: 为什么这件事这么复杂：访存本身在单核上就已经很贵[2]

[1] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

[2] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

因此几乎所有系统都提供某种一致性模型，但为了性能而采取了各种放宽。讲者把本节的核心定为这个取舍。

内存模型是性能与可编程性之间的协商结果

严格模型让程序行为符合直觉，代价是硬件必须频繁等待；宽松模型释放硬件自由度，代价是程序员必须显式表达依赖。因此” 这个程序在多核上对不对” 没有绝对答案，只能针对某个模型回答。这也是为什么同一份代码在不同架构上可能一个正确一个错误。

1.2 本章小结

” 读到最新值” 作为定义不可行，因为访存延迟跨越两个数量级，全局时间没有严格刻度。完全无序又让编程变得不可能。因此每个系统都在两者之间选择一个模型，模型就是对顺序约束的明确规定。

2 用一个实验建立顺序一致性的定义

讲者没有直接给定义，而是用一个观察者实验让定义自然浮现。

2.1 实验设置

程序里有三个线程。线程 0 把偶数依次写入共享变量 x；线程 1 把奇数依次写入同一个 x；线程 2 只是反复采样 x 的值。

问题变成了：线程 2 可能观察到哪些取值序列？哪些组合是非法的？

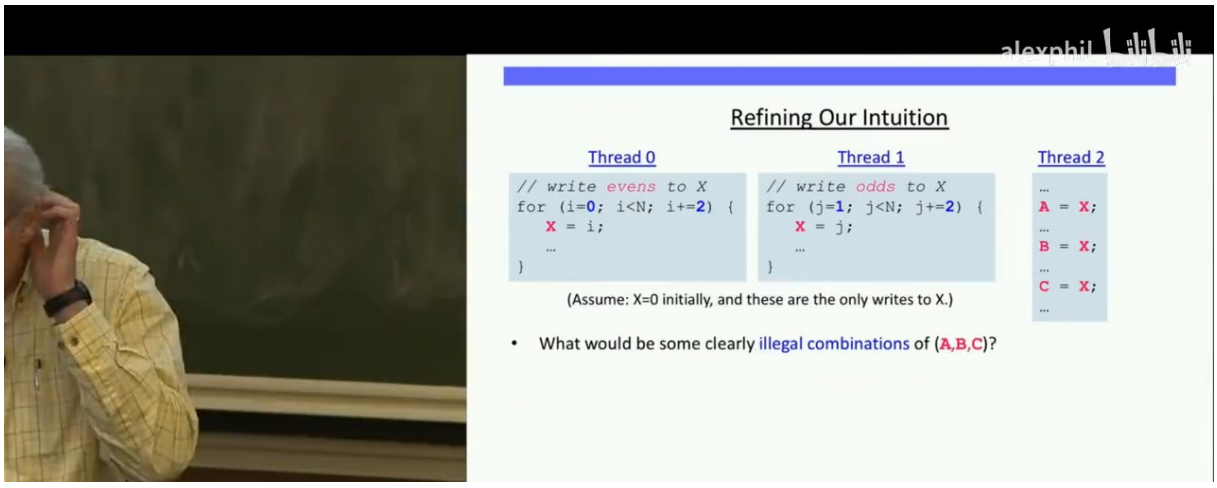


图 3: 细化直觉：线程 0 写偶数、线程 1 写奇数，线程 2 反复采样[3]

2.2 判断规则

讲者给出一个观察序列 4、8、1。学生很快判断这没问题。

理由正是定义的核心：只要偶数之间保持顺序、奇数之间保持顺序，那么这两组数的任意交错都是合法的。因为我们没有对线程 0 与线程 1 的相对速度做任何假设，只要求各自维持自己的顺序。

[3] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

反过来，9、12、3 就有问题——3 不能出现在 9 之后，因为那意味着线程 1 自己的写序被打乱了。

而 7、19、31 是合法的：全是奇数且保持递增。

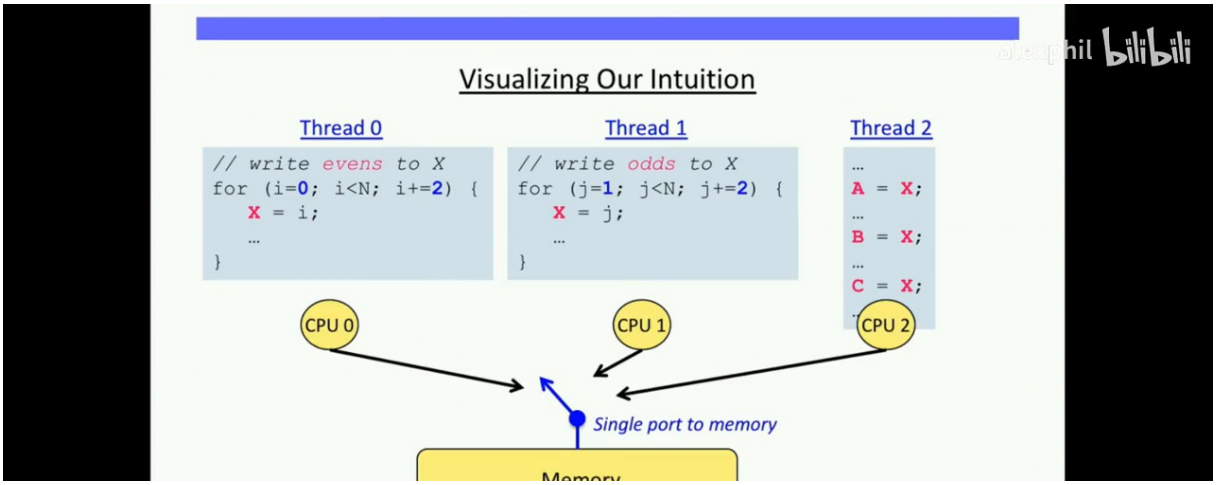


图 4: 可视化直觉：合法交错与非法序列的对比[4]

顺序一致性的判据是可交错性

对任意一个线程自己的写，任何外部观察者都必须按程序中的顺序看到它们；跨线程之间则必须存在某种合法的交错。这个定义不需要引入时间概念，只要求存在一个能解释全部观测的全局序列——这正是它可实现的原因。

2.3 对应的硬件想象

讲者给出一个假想模型来解释这个定义：一块完全全局的内存，没有任何缓存，每次只做一个操作。

每一步，各线程竞争执行自己那条操作的权限，这个” 旋钮” 选中其中一个，把它执行完。于是这个单端口内存就成了一个序列化器，强制把所有线程的操作编织成一个交错序列。

唯一保留的自由度是：我们不对线程之间的相对速度做假设，只要求每个线程内部保持程序序。

[4]视频画面时间区间：00:06:00–00:06:15。若为裁剪图，时间区间仍对应原视频画面。

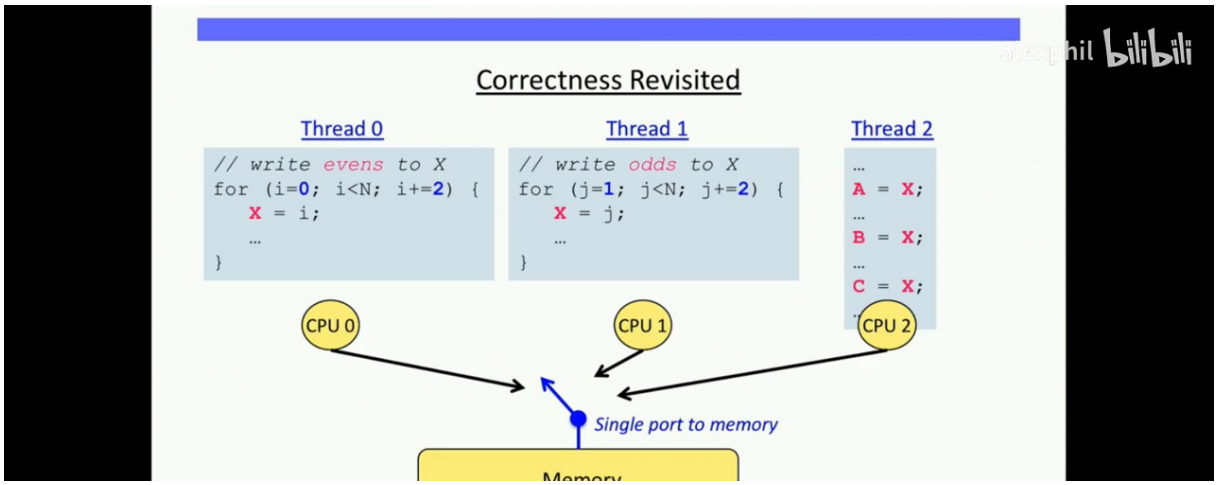


图 5: 重新审视正确性：顺序一致性的判据[5]

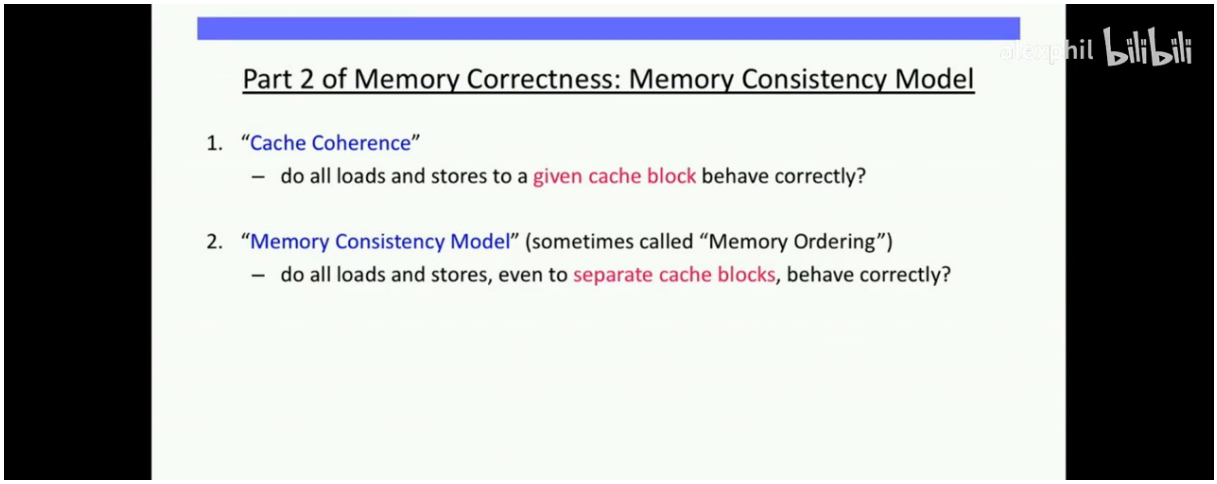


图 6: 内存正确性的第二部分：一致性只覆盖单一变量，模型才覆盖跨地址[6]

Lamport 的顺序一致性定义

顺序一致性要求：每个进程的访存按程序序出现，且所有访存存在一个全局全序，该全序与各进程的程序序一致。讲者的单端口内存模型正是这个定义的操作化形式——把" 存在全序" 变成" 由序列化器逐步产生全序"，从而可以推理。

2.4 本章小结

通过观察者实验，顺序一致性被表述为可交错性：各线程内部保持程序序，跨线程存在合法交错。单端口内存模型给出了它的操作化解释。这个定义只涉及单一变量的取值，尚未触及跨地址问题。

而这正是缺口所在：真实程序里读写不同地址的操作也必须有序。

[5] 视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。

[6] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

3 横跨不同地址：更难的约束

3.1 顺序一致性的判据

把上面的规则推广到多个地址，就是经典的定义：

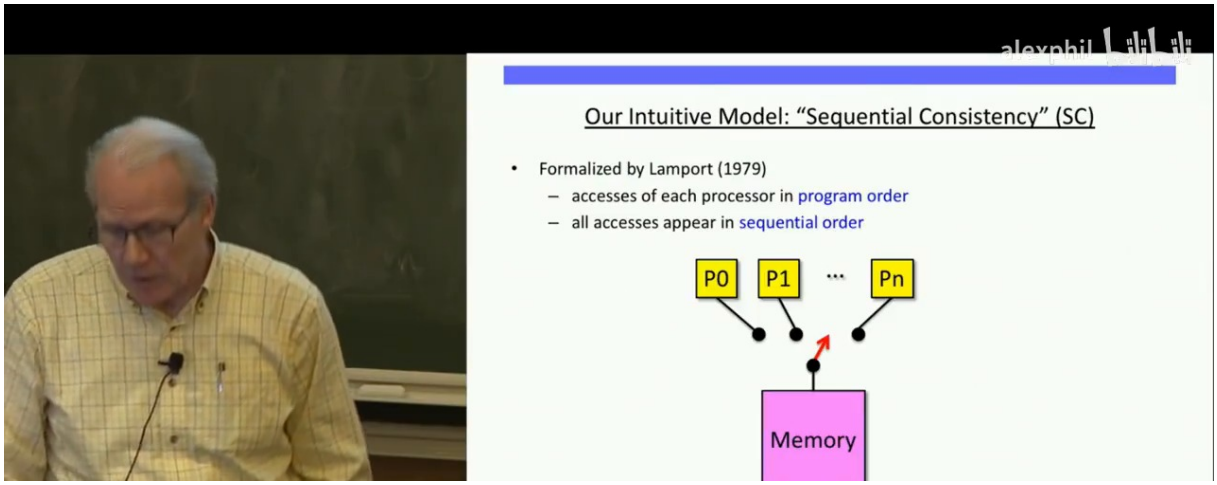


图 7: 直觉模型：顺序一致性（Lamport 1979），各进程访存按程序序[7]

$$\forall p:\ \text{order}_p \text{ 按程序序};\quad \exists \text{ 全序 } \sigma \text{ 与所有 } \text{order}_p \text{ 一致}$$

- order_p：进程 p 的访存序列，必须等于其程序中的顺序；
- σ：一个全局全序，用来解释所有观察者看到的顺序；
- 条件要求 σ 与每个 order_p 相容，即存在（而不要求唯一）这样的全序。

3.2 单核上已经不成立

讲者随即指出关键事实：即便在单处理器上，实现这个” 看起来是顺序执行” 的假象也需要大量硬件支持，而 ** 这些硬件不跨线程生效 **。

也就是说，即使在同一颗处理器上，不同线程的访存实际上可能不按你从程序里看到的顺序发生。

[7]视频画面时间区间：00:26:30–00:26:45。若为裁剪图，时间区间仍对应原视频画面。

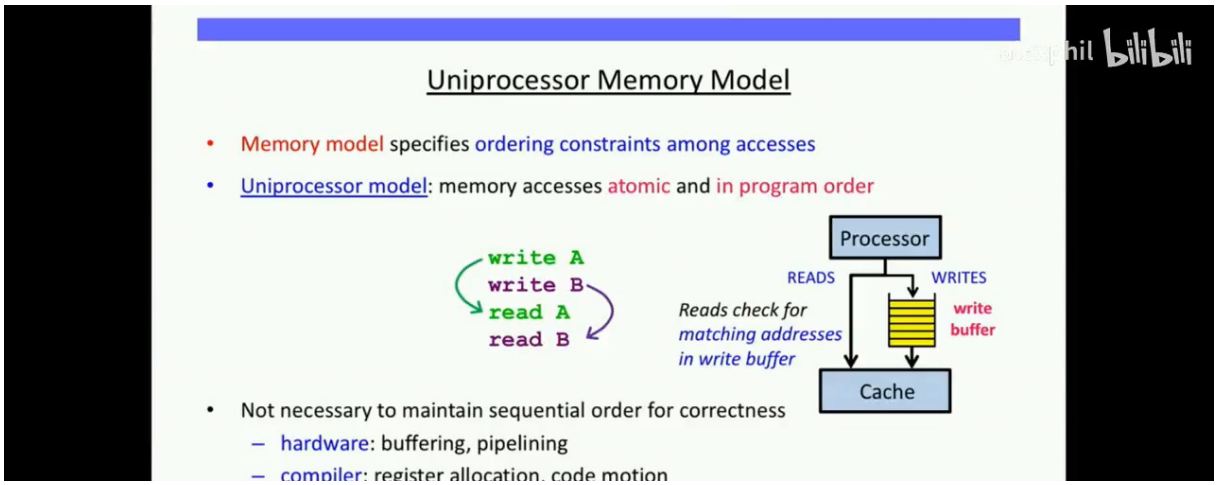


图 8: 单处理器内存模型：规定了访存之间的顺序约束[8]

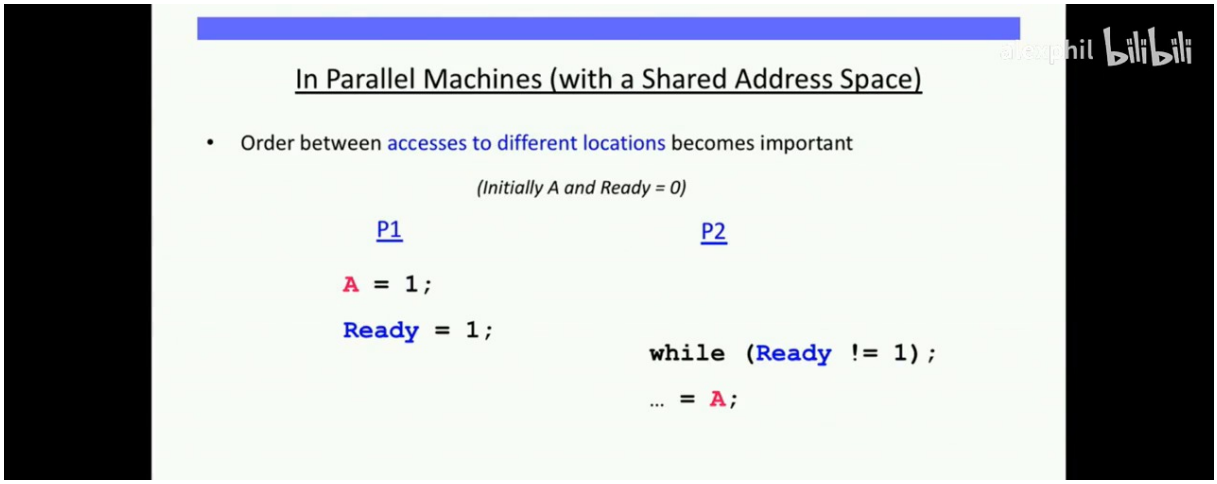


图 9: 共享地址空间并行机器中：不同地址之间的顺序变得重要[9]

3.3 写缓冲区：重排的物理来源

讲者给出机制。程序里写着”先写 A，再读 B”，逻辑上后一条跟着前一条。但硬件会尝试：如果读 B 与写 A 无关，就可以在写 A 完成之前开始读 B。事实上完全可能出现读 B 先完成。

实现这件事的技术叫写缓冲区。处理器说”把这个数据存到内存”，实际只是把它塞进队列，然后就可以认为这次写完成了。之后由包括缓存逻辑在内的硬件负责把它推出去。

[8]视频画面时间区间：00:21:30–00:21:45。若为裁剪图，时间区间仍对应原视频画面。

[9]视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

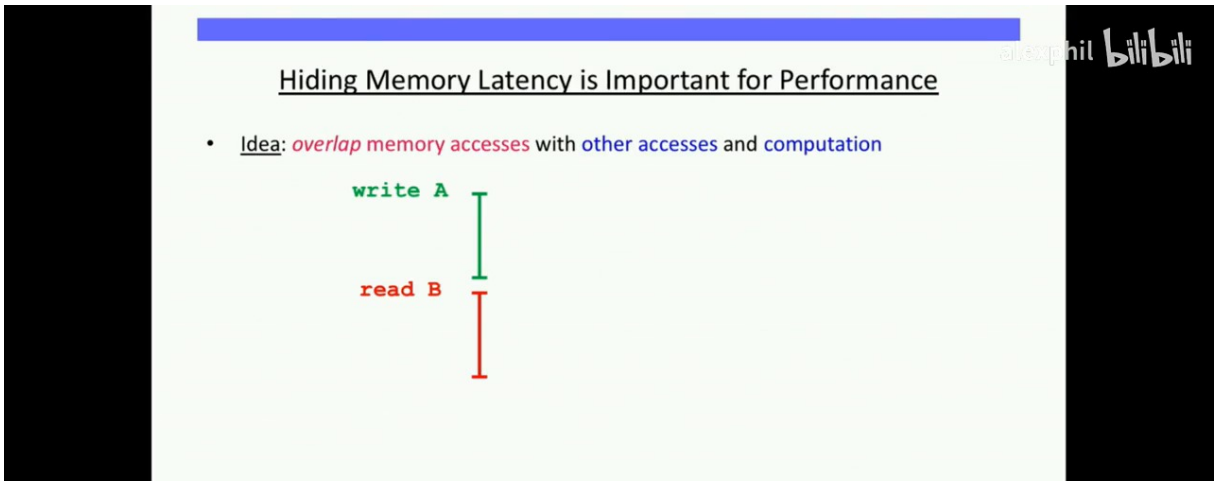


图 10: 隐藏内存延迟对性能很重要：让访存与其他工作重叠[10]

也有少量逻辑保证读写自己的一致性：读操作会先看一眼写缓冲区，若有对同一地址的待定写，就直接从缓冲区取值，而不去缓存读。

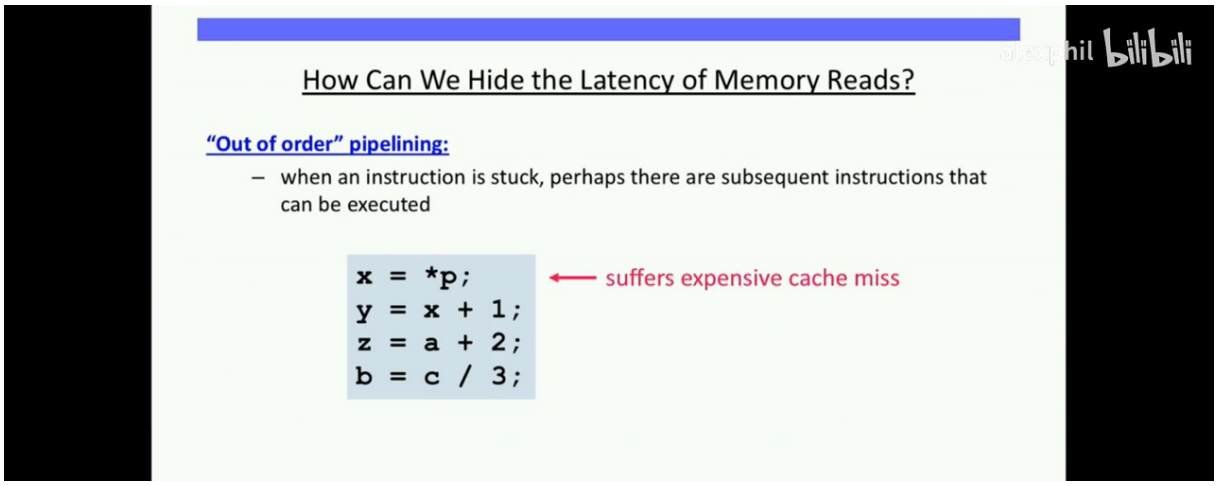


图 11: 如何隐藏内存读的延迟：乱序流水线在指令受阻时继续推进其他指令[11]

但关键在于：队列里可以积累一批还没进入内存系统的写，因此监听总线和缓存协议都不知道它们的存在。

[10] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。
[11] 视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。

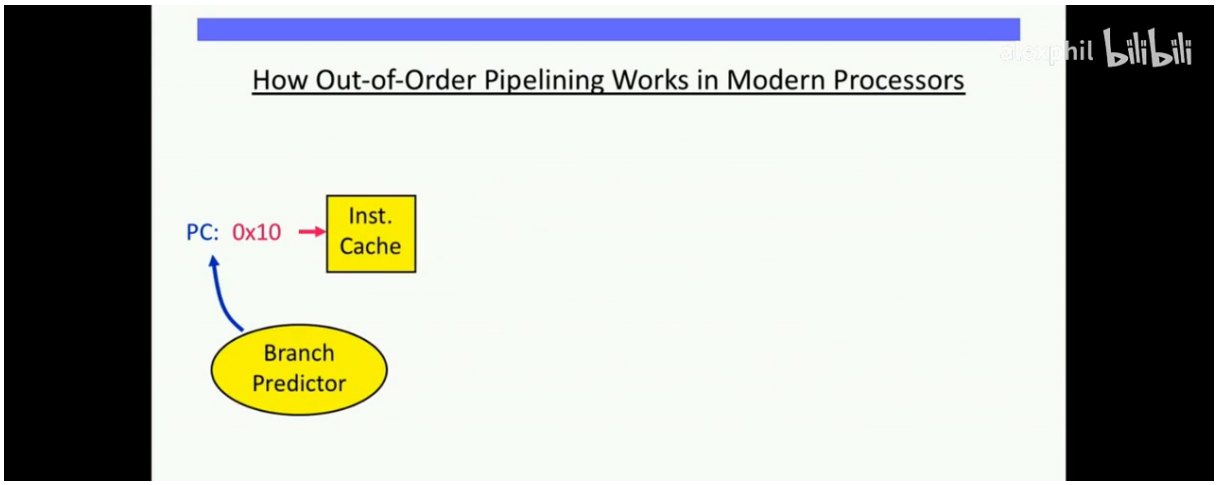


图 12: 现代处理器的乱序流水线：分支预测与乱序发射[12]

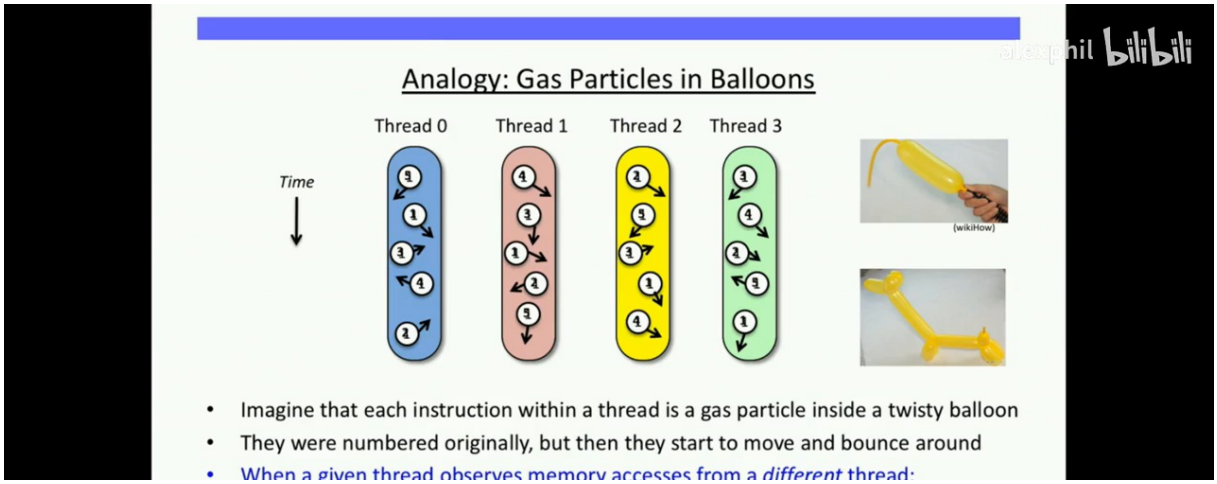


图 13: 类比：气球里的气体分子，各线程独立运动但仍受容器约束[13]

重排不是缺陷而是性能手段

隐藏访存延迟是性能的必要条件。若每条访存都必须等到前一条完成，流水线会在每次缺失时停摆数百周期。写缓冲区、乱序发射、分支预测都是为此存在。因此重排不可能被”修好”，只能被约束——内存模型的作用就是划定哪些重排可以接受。

3.4 本章小结

顺序一致性要求存在一个与各线程程序序相容的全局全序。但这个要求在单核上就已不成立：写缓冲区让写可以”提前完成”，乱序执行让读可以提前开始，而缓存协议完全看不到尚未推出的写。重排是为隐藏延迟而必然存在的。

[12] 视频画面时间区间：00:19:00–00:19:15。若为裁剪图，时间区间仍对应原视频画面。

[13] 视频画面时间区间：00:20:30–00:20:45。若为裁剪图，时间区间仍对应原视频画面。

4 反例：标志位同步为什么会失效

讲者用一个具体模式说明后果。

4.1 经典模式

处理器 1 更新数据 A，然后把标志位置为真。处理器 2 忙等这个标志位，一旦为真就认为可以安全读取 A。

这看起来是跨程序同步的标准做法。但如果读写相对顺序不保证与程序序一致，它就会失效。

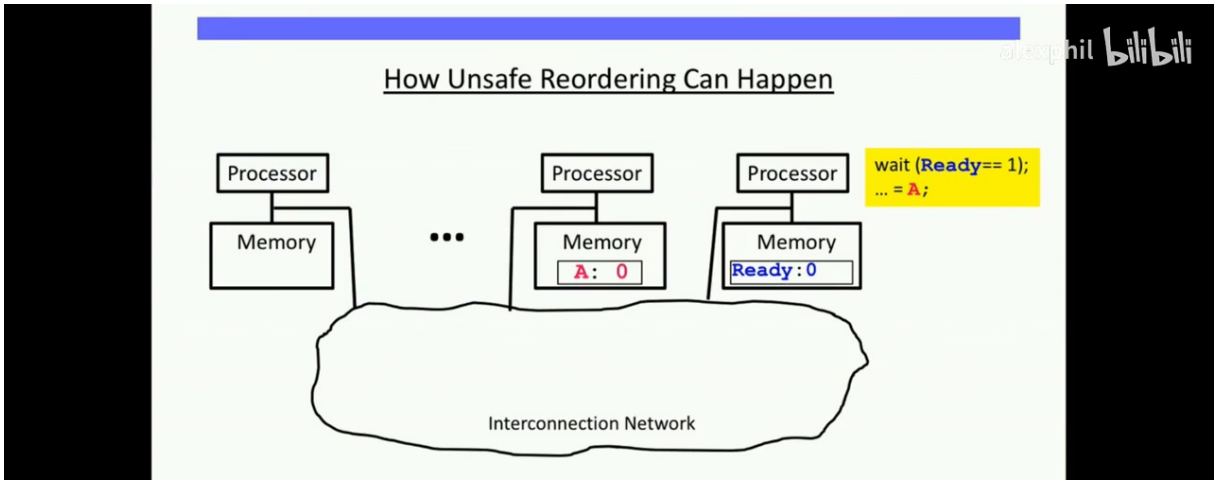


图 14: 不安全的重排如何发生：等待标志位后读到旧数据[14]

讲者描述了时序：处理器 2 持有 ready 的副本并在上面自旋，看到它是 0；处理器 1 把 A 设为 1，发出一系列事务，最终让 A 全局可见；但它设置 ready 为 1 的操作走得更快。

于是处理器 2 可能读到 ready 已经为 1，而去读 A 时拿到的还是旧值。

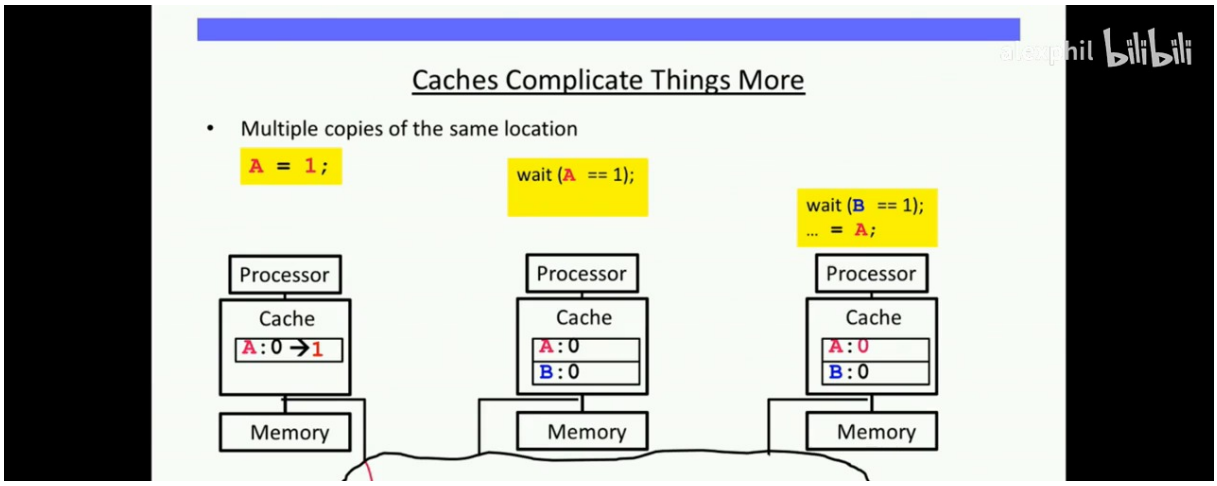


图 15: 缓存让情况更糟：同一位置的多个副本加剧了可见性错位[15]

[14] 视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

[15] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

标志位同步在弱模型上不成立

”先写数据、再置标志”依赖两条写之间的顺序对第三方可见，而这恰恰是模型允许打破的。写缓冲区、多级缓存与互连的延迟差异都可能导致标志先到。正确做法是使用释放/获取语义：写侧以释放方式置标志，读侧以获取方式读取，由硬件与编译器共同保证顺序。

4.2 链式依赖也救不了

讲者进一步给出更复杂的例子：三个线程构成一条依赖链。

左边把 A 设为 1；中间等待 A 变为 1 后把 B 设为 1；右边等待 B 变为 1 后去读 A。

即使有链式依赖，仍然可能出错：第一个线程启动了 A 的更新，很快触发了中间那个线程认为 A 是 1；但 A 的更新真正传遍全系统可能更慢。于是 B 先被置 1 并传到第三个线程，而那时 A 的更新还没到，第三个线程读 A 只能拿到旧值。

讲者总结说，这不只是”这里写那里读”的简单情形，一串有依赖的写读也可能因为延迟差异而拿不到正确值。

4.3 精简成可判定的反例

讲者把问题简化以便判定。去掉自旋，只问：处理器 1 可能读到哪些值组合？其中 (1,0) 这种组合不应出现。

$$(x, y) \neq (1, 0)$$

- x：处理器 1 读到的标志位值；
- y：处理器 1 读到的数据值；
- $(1, 0)$ 表示看到了”数据已就绪”的标志，却读到了初始值，这违背了写方的程序序。

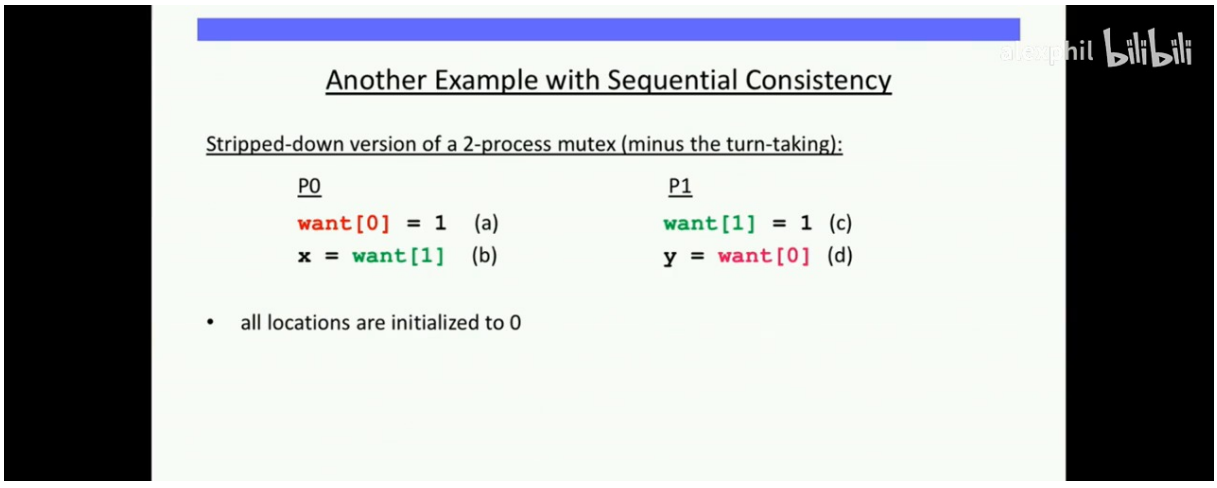


图 16: 互斥协议中不应出现的取值组合：两个进程不可能都读到初始值[16]

[16] 视频画面时间区间：00:29:00–00:29:15。若为裁剪图，时间区间仍对应原视频画面。

互斥协议的例子同理。两个进程各有一个标志，表示自己想进入临界区。若两个进程读到的都是初始值（组合 $(0,0)$），就意味着双方都没看见对方的声明——而实际上至少有一方已经写过。因此 $(0,0)$ 不可能。

讲者说这些都是相当标准的结论。而他提出的问题是：如果完全不能假设顺序一致性，那最基本的同步原语该怎么实现？

4.4 本章小结

标志位同步与互斥协议都依赖跨地址的顺序可见性，而弱模型允许打破它。链式依赖也无法补救，因为延迟差异会沿链累积。(1,0) 与 (0,0) 这类组合成为判断实现是否正确的探测器。

这引出一个两难：既要性能（允许重排），又要正确性（需要顺序）。下面的方案就是在这个张力中找平衡。

5 如何恢复顺序：从严格到逐步放宽

5.1 最严格的做法

一种办法是强制每个处理器同时只能有一个未完成访存。配合单地址的一致性协议，就足以保证每个处理器按程序序执行访存。

讲者说其实不必这么严厉：可以让后来的操作等待前一个被视为”已完成”。

5.2 完成是什么意思

这里有个重要定义。一次访存的完成不是指数据真的到达内存，而是指它已经进入了内存系统的某个位置，从那里保证会被完成。

对写来说，塞进写缓冲区不算完成——它必须已经进入整个缓存协议、开始按要求作废相关的行。这个位置有时叫提交点，也就是说这次写在语义上确实要发生了。

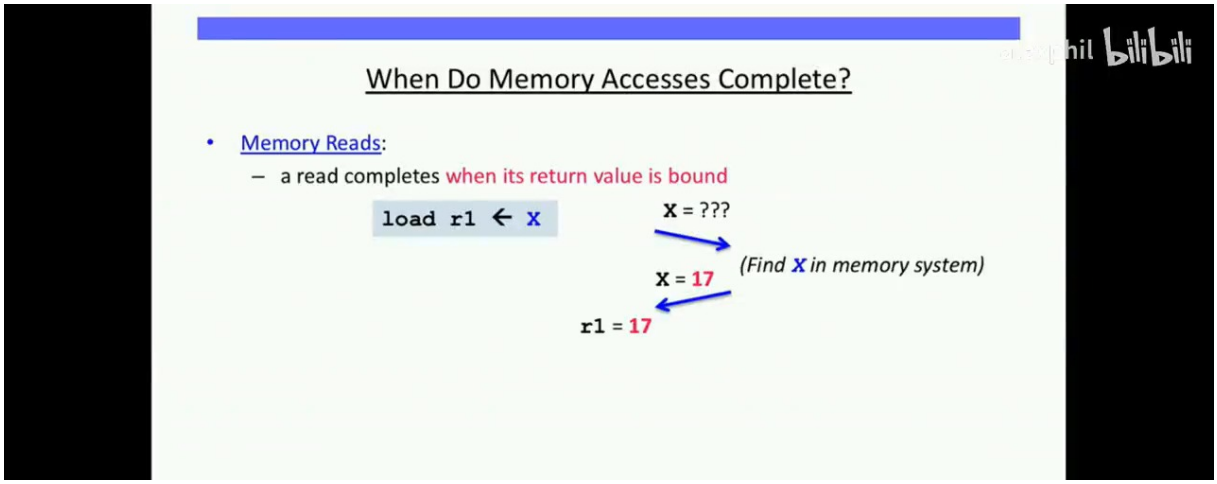


图 17: 访存何时算完成：读在返回值绑定时完成，写在到达提交点时完成[17]

对读来说，一旦确定了它要返回哪个值（无论来自内存还是别的缓存），就可以算完成。

[17] 视频画面时间区间：00:32:00–00:32:15。若为裁剪图，时间区间仍对应原视频画面。

”完成”是一个语义概念而非物理概念

读不需要真的把数值传回寄存器，写也不需要真的更新主存，只要已经推进到协议能保证必然完成的位置即可。这个定义让硬件可以在不牺牲正确性的前提下大幅重叠操作——因为等待的是”承诺”，而不是”结果”。

5.3 严格方案的代价

讲者说这就是在多处理器系统上实现经典顺序一致性的方式，但它是个相当严厉的规则，会让性能为写操作付出很大代价。

把”每次只有一个未完成访存”写成约束，就是：

$$\forall i:\ \mathrm{commit}(op_i) \prec \mathrm{begin}(op_{i+1})$$

- op_i：程序序中第 i 个访存操作；
- $\mathrm{commit}(op_i)$：该操作到达提交点的时刻；
- $\mathrm{begin}(op_{i+1})$：下一个操作可以开始的时刻；
- $\prec$：严格先于。

放宽模型的做法就是允许某些操作对违反这个约束——TSO 允许读越过尚未 commit 的写。形象地说，这相当于在每次访存之间给气球拧一道结：相邻操作之间不能有任何重叠。

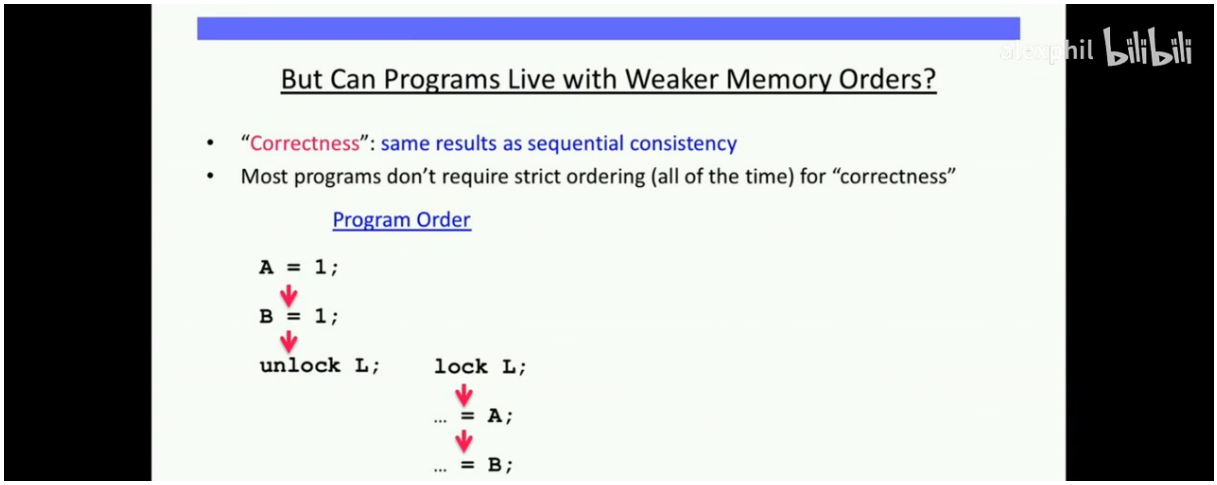


图 18: 程序能否接受更弱的顺序：停顿时间随模型放宽而大幅下降[18]

[18] 视频画面时间区间：00:39:00–00:39:15。若为裁剪图，时间区间仍对应原视频画面。

5.4 TSO：允许读越过写

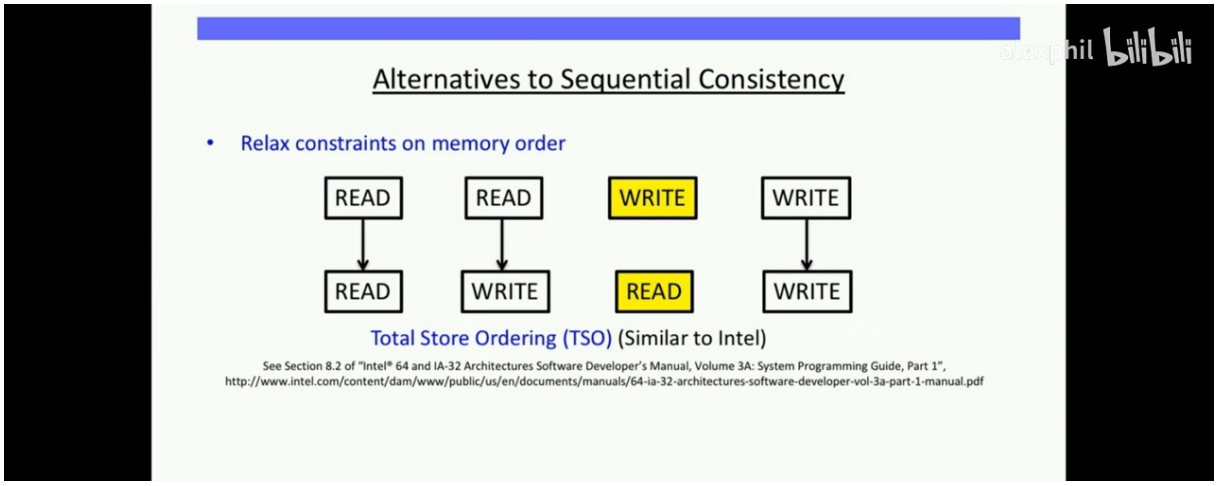


图 19: 顺序一致性的替代方案：放宽访存顺序约束[19]

最简单的放宽叫全存储顺序（TSO）。它允许在一次写尚未完成时就开始读——这正是做写缓冲区的人想要的，因为可以把写结果塞进队列就开始下一条操作。

但它保留了其他三条约束：读与读有序、读与写有序（必须先完成前一个读才能把写放进缓冲区）、写与写有序（用 FIFO 队列保证）。

讲者说这是硬件设计者可以" 不太费力就提供" 的方案，而事实上这正是 Intel 提供的基本语义（细节比这更复杂）。

模型	读-读	读-写	写-写
SC（顺序一致）	保序	保序	保序
TSO	保序	保序	保序（FIFO）
PSO	保序	保序	可重排

表 1: 各模型对访存顺序的约束（放宽的是写之后的操作）

5.5 PSO：连写也放宽

再放宽一步叫部分存储顺序（PSO）：写之后不做任何约束。

这在实现上的含义是写缓冲区不必是 FIFO，写可以任意顺序完成。

讲者给出性能对比：在这些基准上，从顺序一致性到 TSO，用于等待写而停顿的时间大幅减少，因为写被丢进缓冲区就可以先不管。

[19] 视频画面时间区间：00:36:00–00:36:15。若为裁剪图，时间区间仍对应原视频画面。

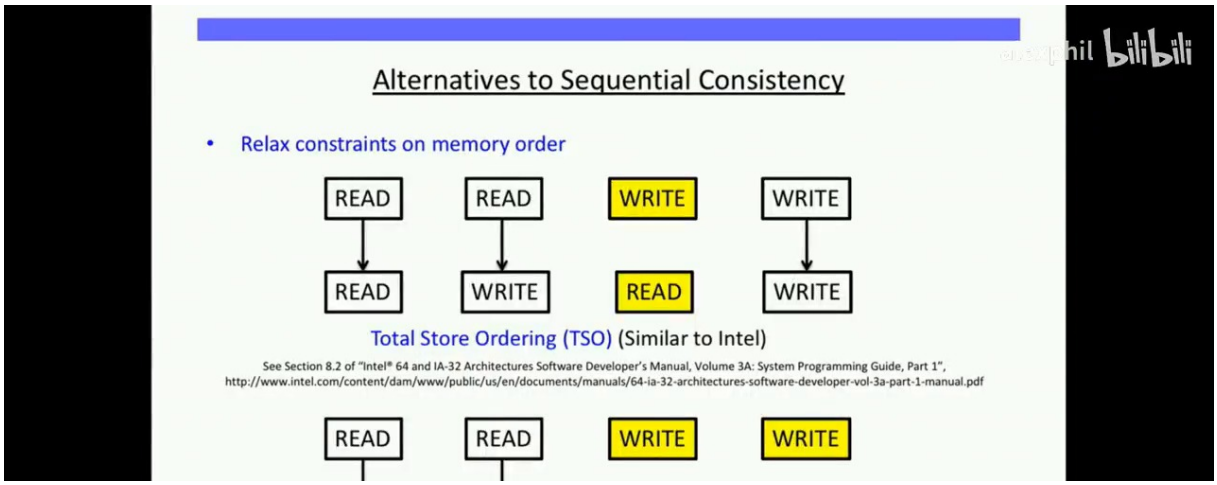


图 20: PSO：连写之间的顺序也放宽，写缓冲区无需 FIFO[20]

5.6 本章小结

恢复顺序的最严格做法是每次只有一个未完成访存，但代价巨大。关键是" 完成" 被定义为到达提交点，而不是物理完成。在此基础上可以逐条放宽：TSO 允许读越过写、保留其余三条约束，基本对应 Intel 语义；PSO 进一步放宽写之间的顺序，停顿时间显著下降。

6 弱序模型与栅栏

6.1 假设程序已正确同步

讲者转向另一条思路：不再要求硬件对所有访存保序，而是假设程序已经用正确的同步手段保护了共享状态。

具体来说，如果存在全局状态，那么除非持有适当的锁，程序不会去修改它。

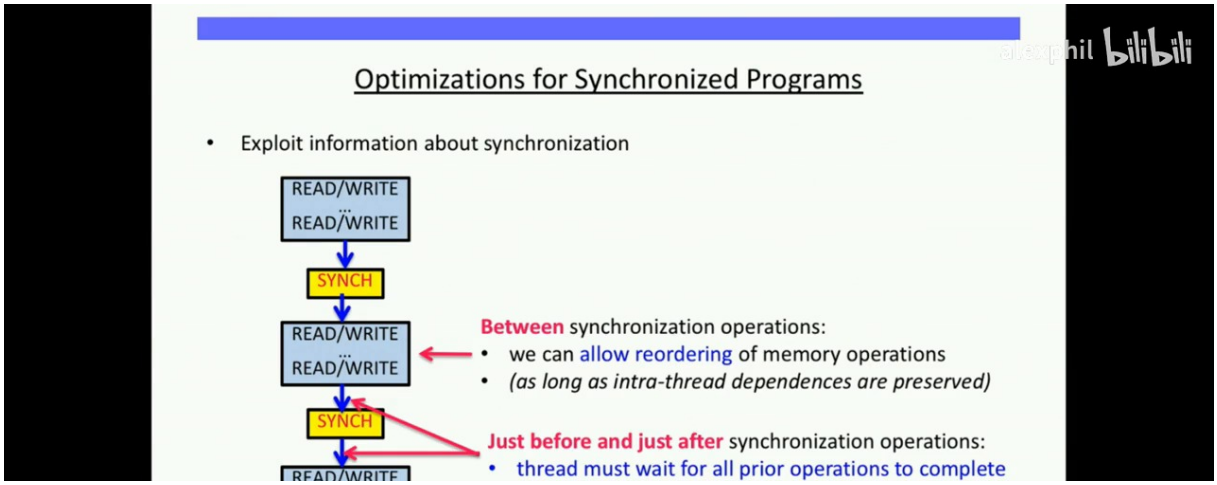


图 21: 为已同步程序做的优化：利用同步信息放宽顺序[21]

[20] 视频画面时间区间：00:38:00–00:38:15。若为裁剪图，时间区间仍对应原视频画面。

[21] 视频画面时间区间：00:47:00–00:47:15。若为裁剪图，时间区间仍对应原视频画面。

一个典型片段是：先做一些计算，然后取锁、更新全局数据结构、释放锁。此后任何操作都应当能看到这次更新。

关键落在两处：临界区内部的访存顺序无关紧要，因为这段代码只有自己能改这些数据；而释放锁之前，所有本该在临界区内完成的更新必须真的生效。

弱序的正确性完全依赖程序自身的同步纪律

硬件不再替你保证跨地址顺序，它把这份责任交给了程序和编译器。因此”我能读懂这段代码的意图”不足以保证正确——必须确认每一处共享访问都在同步保护之下。这也是为什么弱序模型下最简单的数据竞争都可能造成难以复现的错误。

6.2 只读共享数据的额外麻烦

讲者提到一个容易被忽略的细节：通常修改全局结构需要写权限，但程序经常允许其他线程以只读方式访问共享数据。例如一个计数器，更新它的线程必须持锁或用原子操作，但任何人都可以在任意时刻读它。

这就要求更细致地界定：对共享数据的只读访问，我们假设了什么样的顺序？

6.3 MFENCE

讲者给出 Intel 提供的原语。MFENCE 的语义是：到这一点为止所有此前未完成的读写都必须完成，并且在到达它之前不开始任何新操作。

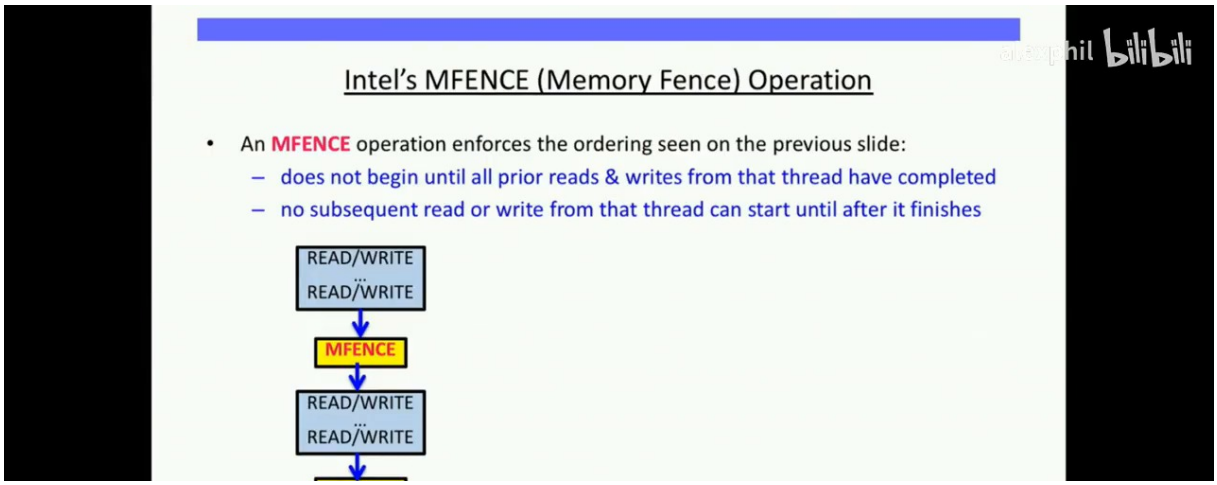


图 22: Intel 的 MFENCE：强制前序读写完成后再继续[22]

逻辑上它就是在气球上拧一道结。同样地，它不要求最终的写真的发生，只要求推进到协议能保证完成的位置。

实现上它并不需要等一切结束：遇到 MFENCE 时线程会停顿，直到写缓冲区排空。

[22] 视频画面时间区间：00:49:00–00:49:15。若为裁剪图，时间区间仍对应原视频画面。

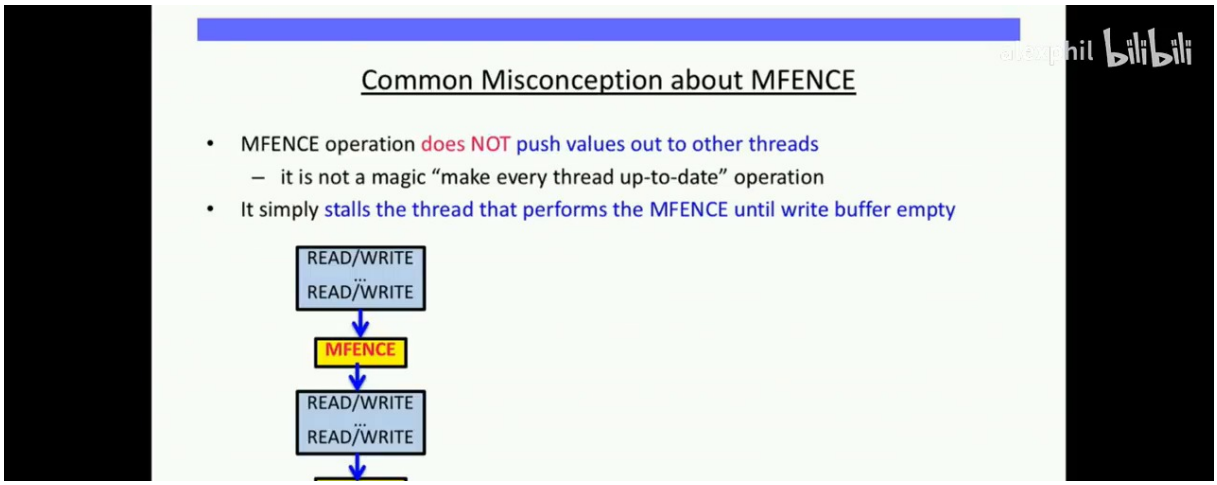


图 23: 关于 MFENCE 的常见误解：它不会把值推给其他线程[23]

讲者还专门纠正了一个常见误解：MFENCE 并不会把值推送到其他线程，它不是”让一切可见”的魔法指令。它只约束本线程的访存顺序。

栅栏约束的是顺序，不是可见性

栅栏保证”此前的操作在可见顺序上先于此后的操作”，但不负责把数据发送出去。数据何时被他人看到，仍由缓存一致性协议决定。混淆这两件事，会导致在需要同步可见性的场合误用栅栏而得不到效果。

6.4 ARM 的现实教训

讲者最后给出一个很有实践价值的提醒。ARM 处理器的内存模型比 x86 宽松得多。众所周知，为它编程要困难得多，但换来的是更高性能。

他的经验说法很有说服力：你的代码在所有你试过的 Intel 处理器上都跑得好，然后你把它移植到 ARM 上，突然就不工作了。

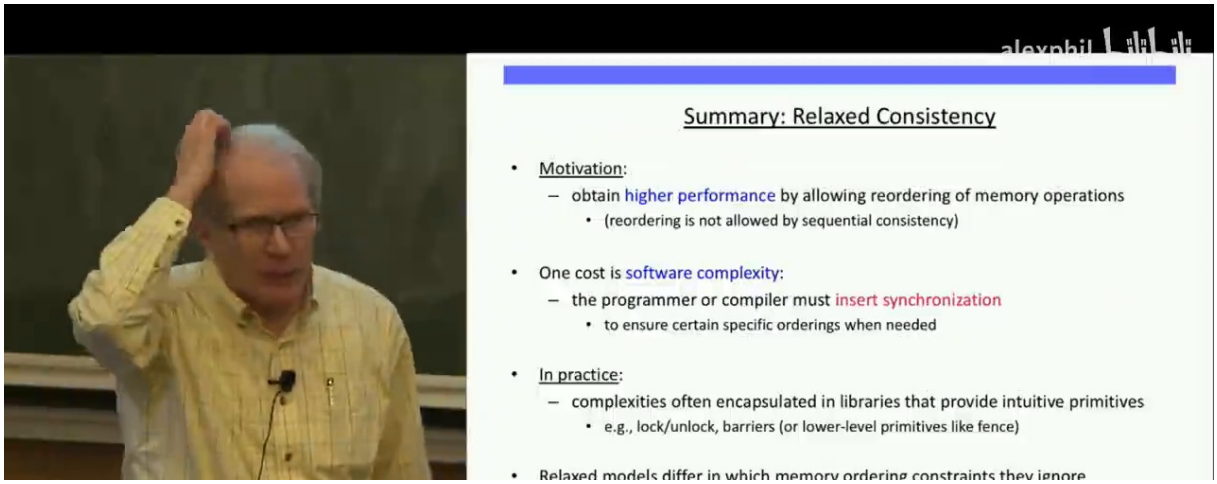


图 24: 放宽一致性的总结：纪律性编程与使用库原语的重要性[24]

[23] 视频画面时间区间：00:50:00–00:50:15。若为裁剪图，时间区间仍对应原视频画面。

[24] 视频画面时间区间：01:08:00–01:08:15。若为裁剪图，时间区间仍对应原视频画面。

由此得到这条建议：编程要有纪律，任何棘手的同步都应当使用现成的库，而不要自己发明。

6.5 本章小结

弱序模型把顺序责任交给程序：临界区内顺序无关，但释放锁前必须让更新生效。它要求区分读写共享数据时的假设，并对只读共享场景格外小心。MFENCE 这类栅栏只约束顺序、不负责可见性。模型越宽松性能越好，但跨架移植的风险越大，因此应尽量使用库提供的同步原语。

7 总结与延伸

讲者以一句话收束：这就是今天全部内容。

把全讲串起来，这是一条从直觉到反例再到工程规则的路径。”读到最新值”作为定义不可行，因为访存延迟跨越两个数量级、全局时间没有精确刻度；完全无序又无法编程。于是每个系统选择一个模型。用观察者实验可以把顺序一致性表述为可交错性：各线程内部保持程序序，跨线程存在合法交错，单端口内存是它的操作化模型。但一旦涉及不同地址，这个要求在单核上就已不成立：写缓冲区让写”提前完成”、乱序执行让读提前开始，而缓存协议看不到尚未推出的写。于是标志位同步失效，链式依赖也无法补救，(1,0) 与 (0,0) 成为可判定的错误探测器。恢复顺序的办法是把”完成”定义为到达提交点，并逐条放宽约束：TSO 允许读越过写，PSO 进一步放宽写序，停顿时间显著下降。另一条路是弱序模型，把顺序责任交给程序的同步纪律，并用 MFENCE 这类栅栏在关键处恢复顺序。最后，ARM 比 x86 更宽松，代码在 Intel 上正常不代表在 ARM 上正确，因此应当使用库提供的原语。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 一致性模型不是可有可无的抽象层，它决定了哪些代码写法是可移植的。同一份”看起来正确”的代码在 SC、TSO、ARM 上的表现可以完全不同。因此写并发代码时，目标模型必须作为前提明确下来。
- ”完成”的语义化定义是整套放宽方案的技术枢纽。正因为完成指到达提交点而非物理到达，硬件才能在不破坏模型的前提下重叠操作。理解这一点，才能理解为什么栅栏不必等到写真的落到内存。
- 栅栏与原子操作解决的是不同问题。栅栏约束顺序，原子操作保证不可分割，缓存协议保证单一地址的一致。缺任何一环，同步都可能失效，而三者都齐备时才等价于顺序一致性下的行为。
- 平台差异是实践中的主要风险来源。x86 的 TSO 相对宽松程度有限，很多错误写法恰好在它上面工作；ARM 的弱模型会把这些错误暴露出来。因此”在我机器上是对的”是并发编程中最危险的判断依据之一。

这堂课的边界在于它是概念导论：讲者明确表示内容不会出现在紧接着的考试里，而模型的形式化规则、各架构的具体条文都需要专门查阅。因此本文给出的是判断框架与陷阱地图。

两个可以直接从本讲数据提出的下一步问题：其一，若一份代码要在 x86 与 ARM 上都正确且高效，应当按最强的 SC 写还是按最弱的模型写，两者在性能与可维护性上如何取舍？其二，MFENCE 只排空写缓冲区，那么在一个需要”让其他线程看到我的更新”的场景里，还缺少哪一环，应当如何配合原子操作完成？